

Потоки ввода и вывода

Вот максимальная таблица по потокам, FD, перенаправлениям, POSIX, ядру и Bash — всё в одном месте для твоей абстракции. Продолжение по `/dev/` файлам, буферизации и практическим паттернам ниже.

📁 Специальные файлы `/dev/` (продолжение)

Файл	FD	Чтение	Запись	Bash	Особенности
<code>/dev/urandom</code>	—	Псевдослучайные байты (не блокирует)	—	<code>head -c 32 /dev/urandom base64</code>	Для сгенерации UUID. Криптографически безопасный.
<code>/dev/full</code>	—	EOF	Всегда ENOSPC (ошибка диск полон)	<code>cmd > /dev/full</code>	Тестирует, обрабатывает запись.
<code>/dev/tty</code>	—	Текущий управляющий терминал	Текущий управляющий терминал	<code>echo hi > /dev/tty</code>	Игнорирует перенаправление. Полезно для диагностики.
<code>/dev/fd/N</code>	N	FD N процесса	FD N процесса	<code>cat /dev/fd/0</code>	Linux-only. Аналог <code>/proc/self/fd/N</code> .
<code>/dev/pts/N</code>	—	Псевдотерминал slave	Псевдотерминал slave	<code>screen</code> , <code>tmux</code>	Каждый терминал — отдаленный.
<code>/dev/ptmx</code>	—	Мастер псевдотерминала	Мастер псевдотерминала	—	<code>open(2)</code> для получения мастера. <code>unlockpt()</code> для <code>pts</code> .

🔄 Буферизация stdio (libc)

Поток	Режим по умолчанию	Когда сбрасывается	Как изменить
<code>stdin</code>	Полная (full)	<code>\n</code> или заполнение буфера	<code>setvbuf(stdin, buf, _IOLBF, size)</code>
<code>stdout</code>	Построчная (line) при терминале	<code>\n</code> или <code>fflush()</code>	<code>setvbuf(stdout, NULL, _IONBF, 0)</code>
<code>stderr</code>	Полная (full) при pipe/файле	<code>fflush()</code> или заполнение	<code>setbuf(stdout, NULL)</code> — отключить

Поток	Режим по умолчанию	Когда сбрасывается	Как изменить
<code>stderr</code>	Отключена (unbuffered)	Немедленно	Не меняется (по стандарту)

Bash-эквиваленты:

```

stdbuf -oL cmd      # stdout line-buffered
stdbuf -o0 cmd      # stdout unbuffered
unbuffer cmd        # через exect, полностью отключить буферизацию

```

Практические паттерны для твоей абстракции

Задача	Bash	C	Твой ЯП (идея)
Простой вывод	<code>echo hello</code>	<code>printf("hello\n")</code>	<code>print("hello")</code>
Вывод без <code>\n</code>	<code>printf 'hello'</code>	<code>printf("hello")</code>	<code>print("hello", end="")</code>
Форматированный вывод	<code>printf '%04d' 7</code>	<code>printf("%04d", 7)</code>	<code>print(7, format="%04d")</code>
Перенаправить stdout	<code>cmd > file</code>	<code>freopen("file", "w", stdout)</code>	<code>redirect_stdout("file")</code>
Перенаправить stderr	<code>cmd 2> file</code>	<code>dup2(fd, STDERR_FILENO)</code>	<code>redirect_stderr("file")</code>
Объединить stdout+stderr	<code>cmd > file 2>&1</code>	<code>dup2(STDOUT_FILENO, STDERR_FILENO)</code>	<code>redirect_all("file")</code>
Тихий режим	<code>cmd > /dev/null 2>&1</code>	<code>freopen("/dev/null", "w", stdout)</code>	<code>silence()</code>
Pipe между командами	<code>cmd1 \ cmd2</code>	<code>pipe() + dup2() + fork()</code>	<code>cmd1.pipe(cmd2)</code>
Чтение из строки	<code>cmd <<< "input"</code>	<code>fmemopen()</code>	<code>cmd.input("input")</code>
Process substitution	<code>diff <(sort a) <(sort b)</code>	<code>pipe() + fork() + dup2()</code>	<code>diff(sort(a), sort(b))</code>
Сохранить stdout	<code>exec 3>&1</code>	<code>int saved = dup(STDOUT_FILENO)</code>	<code>save_stream(stdout)</code>
Восстановить stdout	<code>exec 1>&3</code>	<code>dup2(saved, STDOUT_FILENO)</code>	<code>restore_stream(saved)</code>
Закрыть FD	<code>exec 3>&-</code>	<code>close(3)</code>	<code>close_fd(3)</code>

Задача	Bash	C	Твой ЯП (идея)
Неблокирующий ввод	—	<code>fcntl(0, F_SETFL, O_NONBLOCK)</code>	<code>set_nonblocking(stdin)</code>
Ждать готовности	—	<code>poll()</code> / <code>select()</code>	<code>wait_readable(fd, timeout)</code>

Архитектура абстракции для твоего ЯП

Твой ЯП (синтаксис)
<code>print("hello")</code>
<code>redirect_stdout("log.txt")</code>
<code>pipe(cmd1, cmd2)</code>
Runtime / Компилятор (Bash/C/Zig)
Генерация: <code>printf '%s\n' 'hello'</code>
Генерация: <code>exec 1>log.txt</code>
Генерация: <code>cmd1 cmd2</code>
POSIX Shell / Bash
<code>exec</code> , <code>dup2</code> , <code>pipe</code> , <code>open</code> , <code>close</code>
Ядро Linux
VFS → struct file → f_op → драйвер

Чек-лист для безопасной генерации Bash

Правило	Почему	Пример опасного
Всегда <code>printf '%s\n' "\$var"</code>	<code>echo</code> ломается на <code>-n</code> , <code>-e</code>	<code>echo "\$var"</code> где <code>var='-n'</code>
Всегда quote аргументы	Word splitting, glob expansion	<code>echo \$var</code> где <code>var='a b'</code> → два аргумента
Никогда не вставлять данные в format string	Format string vulnerability	<code>printf "\$var"</code> где <code>var='%s%s%s'</code> → crash
Использовать <code>exec</code> для перенаправлений	Применяется к текущему shell	<code>>file</code> без <code>exec</code> — только для команды
Проверять порядок <code>2>&1</code>	<code>2>&1 >file</code> ≠ <code>>file 2>&1</code>	<code>cmd 2>&1 >file</code> — stderr в терминал
Закрывать временные FD	Утечка FD → <code>EMFILE</code>	<code>exec 3>&1</code> без <code>3>&-</code>

Если нужно — могу добавить раздел по **Windows-эквивалентам** (`HANDLE`, `ReadFile`, `WriteFile`, `CreatePipe`, `OVERLAPPED`), или по **WASI** (`fd_read`, `fd_write`, `path_open`), чтобы твоя абстракция была действительно кросс-платформенной.